VELLORE INSTITUTE OF TECHNOLOGY

School of Computer Science and Engineering

Fall Semester 2026–27

BCSE307P · Compiler Design Lab

Slot L49+L50 · Class Number VL2026270103362

ICG WITH BACKPATCHING

One-Pass Intermediate Code Generation for
Control-Flow Statements using Flex and Bison

Assessment 9 · Project Review 2

Implementation Progress Report

Team 15

Prithvi Raghu	24BCE2624
Jai Narayan Jayaprakash Jayanthi	24BCE2751
Divyanshu Pal	24BCE2856

Faculty: Dr. R. Naga Priyadarsini

16 October 2026

Contents

1 Introduction **1**

1.1 Project overview 1

1.2 Objectives 1

1.3 Scope of Review 2 1

2 Background and Design **3**

2.1 Related approaches and the gap 3

2.2 Architecture 3

2.3 Translation scheme for the Review 2 constructs 4

2.4 Data structures 5

3 Implementation Progress **6**

3.1 if-else and the dangling else 6

3.2 while loops 7

3.3 Short-circuit boolean conditions 8

3.4 Nesting 9

3.5 Error reporting 10

4 Novelty: Making Backpatching Visible **11**

5 Testing **14**

5.1 Build 14

5.2 Regression suite 14

5.3 Stress programs 16

6 Progress Summary and Plan for Review 3 **17**

6.1 Progress against the plan 17

6.2 Plan for Review 3 (23 October) 17

References **18**

List of Figures

Figure 2.1	Architecture: one pass, all helpers inside parser.y	4
Figure 3.1	An else-if ladder with compound conditions (examples/e1_ladder.src)	7
Figure 3.2	A while loop printed as a quadruple table (tests/04_while.src)	8
Figure 3.3	Negated disjunction inside a conjunction (examples/e2_bool.src)	8
Figure 3.4	if-else inside if-else inside while (tests/06_nested.src)	9
Figure 3.5	Syntax, semantic and lexical errors with line numbers and exit codes	10
Figure 4.1	Trace of a while loop with a break (examples/e5_trace.src)	12
Figure 5.1	Clean build	14
Figure 5.2	Regression run	16

List of Tables

Table 1.1	Review schedule presented at Review 1	2
Table 1.2	Evaluation criteria for Project Review 2	2
Table 2.1	Three ways to resolve forward jumps	3
Table 2.2	Semantic actions implemented for Review 2	4
Table 2.3	Data structures in parser.y	5
Table 3.1	Errors detected by the generator	10
Table 4.1	Life of each jump in Figure 4.1	13
Table 5.1	Regression tests for the Review 1 and Review 2 scope	15
Table 5.2	Stress programs for the Review 2 constructs	16
Table 6.1	Implementation status at Review 2	17

1 Introduction

1.1 Project overview

Intermediate code generation turns a parsed program into a simple, machine-independent form such as three-address code. When this is done inside the semantic actions of an LR parser, code is emitted while the program is still being read, and control-flow statements create a difficulty: a jump must often be emitted before the instruction it jumps to exists. In `if (a < b) y = c;` the jump taken when the condition is false must land after `y = c`, but when the condition is reduced that instruction has not been generated.

This project builds a one-pass intermediate code generator for a small C-like language using Flex and Bison, and solves the forward-jump problem by *backpatching* (Aho et al. [1], sec. 6.7): every jump with an unknown target is emitted empty, its instruction number is kept in a list, and the whole list is filled the moment the target becomes known. No syntax tree is built and there is no label-resolution pass.

1.2 Objectives

1. **Correct three-address code** for assignments, arithmetic and all structured control flow, nested to any depth, with no jump left unfilled.
2. **A single pass:** code is emitted while parsing and every jump target is filled by backpatching.
3. **Short-circuit booleans:** `&&`, `||` and `!` compile to jumps, so the right operand is skipped once the result is known.
4. **Clear error reporting:** lexical and syntax errors stop with their line number instead of crashing.

1.3 Scope of Review 2

At Review 1 (9 October) the lexer, the complete grammar, the quadruple table and the patch-list helpers were in place, and code generation worked for assignments and the plain `if` statement. The plan presented then set the following scope for Review 2.

Table 1.1 Review schedule presented at Review 1

Milestone	Date	Scope
Review 1	9 Oct 2026	Lexer, full grammar, quad table, patch lists; assignment and `if`
Review 2	**16 Oct 2026**	**`if-else`, `while`, `&& || !`, nesting, error reporting; `--trace` mode**
Review 3	23 Oct 2026	`do-while`, `for`, `break` / `continue`; step-by-step trace; two-pass comparison; manuscript
Submission	30 Oct 2026	Final report and presentation uploaded to VTOP

Table 1.2 maps the evaluation criteria of Assessment 9 to the parts of this report that address them.

Table 1.2 Evaluation criteria for Project Review 2

Criterion	Marks	Addressed in
75% of implementation / prototype progress	5	Chapter 3 (implementation), Chapter 5 (testing), Chapter 6 (progress)
Novelty	2	Chapter 4 (trace mode)
Documentation / progress report	1	This report
Presentation and Q&A	2	Review presentation; figures in this report are taken from the live tool

2 Background and Design

2.1 Related approaches and the gap

Compilers resolve forward jumps in three main ways. Table 2.1 summarises them from the literature survey presented at Review 1.

Table 2.1 Three ways to resolve forward jumps

	Two-pass, symbolic labels	**Backpatching (this project)**	**Basic-block IR**
Passes over the code	2: emit with labels, then resolve	1: targets filled during parsing	1 to build blocks; targets are block references
Held until targets are known	Label table and all code	Lists of unfilled jump numbers	Block objects and control-flow graph
Fit with Bison	Easy, but needs a resolution pass	Natural: empty marker rules capture the next instruction number	Heavy: needs an IR framework
Main source	Appel [2]	Aho et al. [1]	Lattner and Adve [5]

Backpatching is standard textbook material, but its patch lists are invisible: a student sees the finished quadruples, not the moment each jump was created empty and later filled. Few student tools show these lists while code is generated. This gap is the basis of the project's novelty (Chapter 4).

2.2 Architecture

The generator is a single pass. The Flex lexer turns source text into tokens and tracks line numbers. The Bison parser recognises the grammar, and the semantic action of each rule emits quadruples the moment the rule is reduced. The actions call helper groups that live below the grammar in the same file.

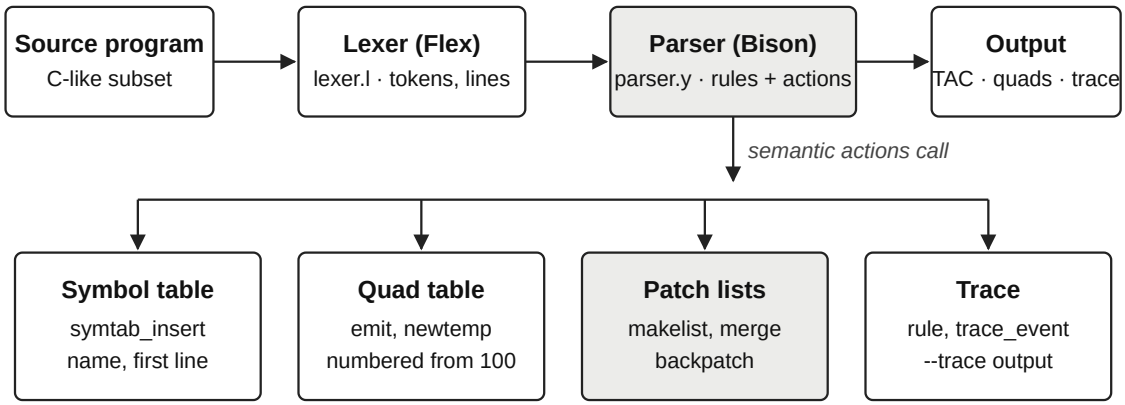


Figure 2.1 Architecture: one pass, all helpers inside `parser.y`

2.3 Translation scheme for the Review 2 constructs

Every non-terminal that produces jumps carries patch lists. For a condition `B`, `B.truelist` and `B.falselist` hold the jumps to be sent where control goes when `B` is true or false. For a statement `S`, `S.nextlist` holds the jumps that must go to the code after `S`. `nextinstr` is the number the next quadruple will receive.

Table 2.2 Semantic actions implemented for Review 2

Production	Semantic action
`S → if ( B ) M1 S1 else N M2 S2`	backpatch(B.truelist, M1.instr); backpatch(B.falselist, M2.instr) S.nextlist = merge(S1.nextlist, N.nextlist, S2.nextlist)
`S → while M1 ( B ) W S1`	backpatch(S1.nextlist, M1.instr); backpatch(B.truelist, W.instr) emit(goto M1.instr); S.nextlist = B.falselist
`B → B1 || M B2`	backpatch(B1.falselist, M.instr) B.truelist = merge(B1.truelist, B2.truelist); B.falselist = B2.falselist
`B → B1 && M B2`	backpatch(B1.truelist, M.instr) B.truelist = B2.truelist; B.falselist = merge(B1.falselist, B2.falselist)
`B → ! B1`	B.truelist = B1.falselist; B.falselist = B1.truelist
`B → true | false`	truelist (or falselist) = makelist(nextinstr); emit(goto _)
`L → L1 M S`	backpatch(L1.nextlist, M.instr); L.nextlist = S.nextlist
`M → ε`	M.instr = nextinstr
`N → ε`	N.nextlist = makelist(nextinstr); emit(goto _)
`W → ε`	W.instr = nextinstr; open a loop entry

2.4 Data structures

Table 2.3 Data structures in `parser.y`

Structure	Fields	Functions
Quadruple	kind, op, arg1, arg2, result, target (−1 until filled)	`emit`, `emit_goto`
Quad table	Growable array; `nextinstr` starts at 100	`print_tac`, `print_quads`
Patch list	Array of quad numbers; `NULL` is the empty list	`makelist`, `merge`, `backpatch`
Symbol table	Name and first-seen line	`symtab_insert`
Temporaries	Counter producing `t1`, `t2`, ...	`newtemp`

The Bison `%union` carries `addr` (the name or temporary holding the value of `E`), `b` (the true and false lists of `B`), `nextlist` (for `S`, `L` and `N`) and `instr` (for `M` and `W`). `backpatch` refuses to fill a quadruple that is not an unfilled jump and stops with an internal error, so a wrong semantic action fails immediately instead of producing plausible but wrong code.

3 Implementation Progress

This chapter presents each Review 2 construct with the rule that implements it and a run of the generator on a program that exercises it. The programs in `examples/` were written to be harder than the regression tests: each output below was traced by hand, jump by jump, before being accepted.

3.1 if-else and the dangling else

An `if-else` needs three kinds of jump: true exits of the condition go to the then-branch, false exits go to the else-branch, and the end of the then-branch must jump over the else-branch. The last jump comes from the marker `N`, which emits an unfilled `goto` and returns it as a one-element list.

The grammar is ambiguous when one `else` follows two `if`s. The declarations `%precedence LOWER_THAN_ELSE` and `%precedence ELSE` make Bison prefer shifting `else`, so it binds to the nearest `if`. The textbook places `N` before `else`; in Bison that produced a reduce/reduce conflict, which precedence cannot resolve. Placing `N` after `else` turns it into a shift/reduce choice, and the emitted `goto` still lands directly after the then-branch because nothing is emitted while `else` is shifted. `%expect 0` makes the build fail if any conflict reappears.

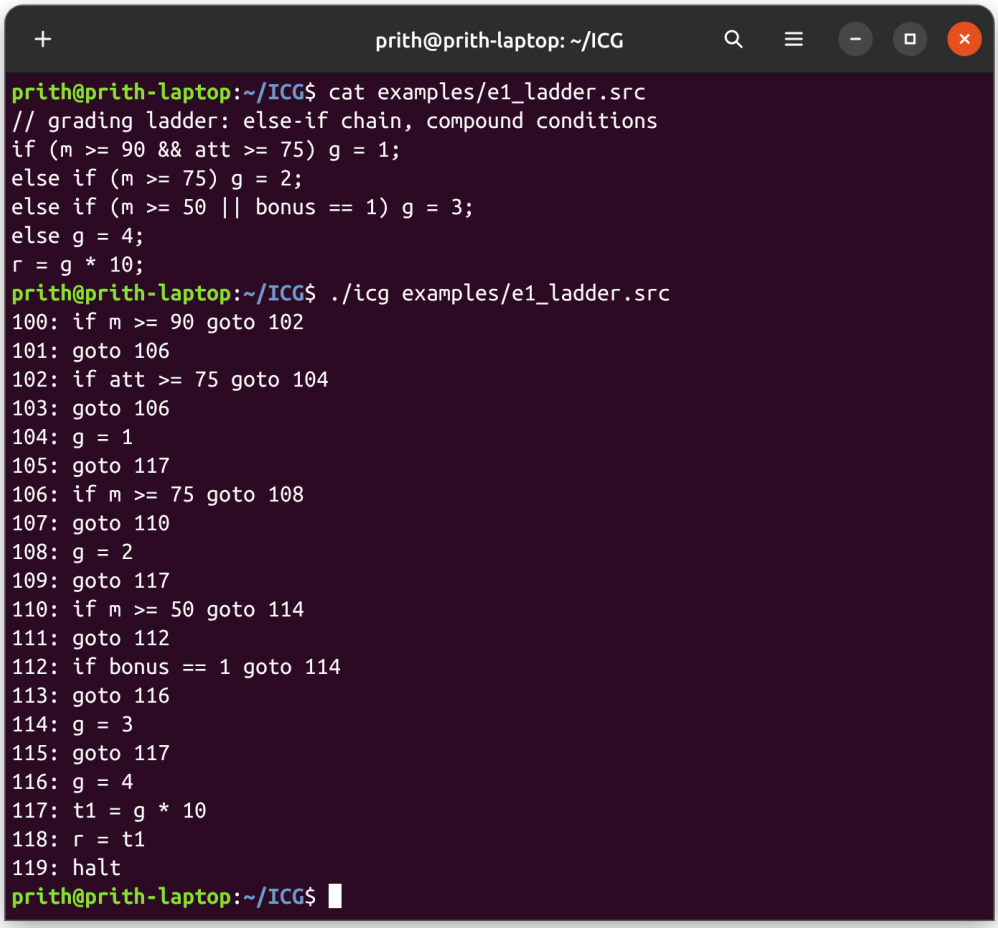


Figure 3.1 An else-if ladder with compound conditions (`examples/e1_ladder.src`)

Each `else if` is an `if-else` nested in the else-branch of the previous one, so Figure 3.1 applies the rule four levels deep. Quads 105, 109 and 115 are the `N` jumps over the remaining branches. Their target is the statement after the whole ladder, so they stay unfilled until `r = g * 10` is reached and are then filled together with 117. In the third condition a true `m >= 50` jumps straight to 114 and `bonus == 1` is never tested.

3.2 while loops

A `while` loop needs one backward jump, from the end of the body to the test, whose target is already known when it is emitted. Such jumps use `emit_goto`, which writes the target immediately and never puts the jump on a patch list. The false exits of the condition become the loop's `nextlist`. Figure 3.2 shows a loop in the quadruple-table format, where jumps appear as `j<` and `j` with the target in the result column.

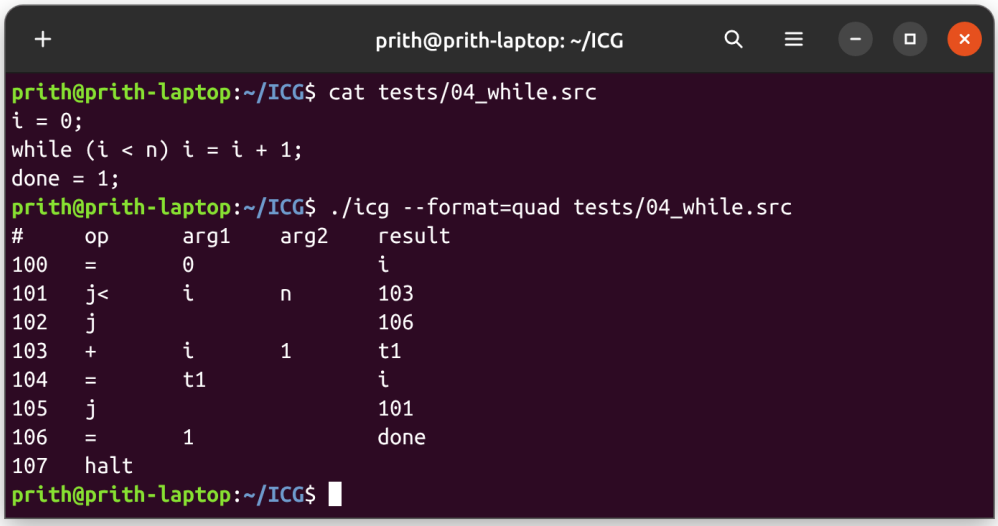


Figure 3.2 A while loop printed as a quadruple table (`tests/04_while.src`)

Quad 105 is the backward jump to the test at 101. Quad 102, the false exit, is filled with 106 only when the next statement `done = 1` begins.

3.3 Short-circuit boolean conditions

Boolean operators generate no arithmetic. Each one only rearranges the true and false lists of its operands (Table 2.2), so the right operand is evaluated only when the left one has not already decided the result. `!` emits nothing at all: it exchanges the two lists. Precedence, from lowest to highest, is `||`, `&&`, `!`, relational operators, `+ -`, `* /` and unary minus.

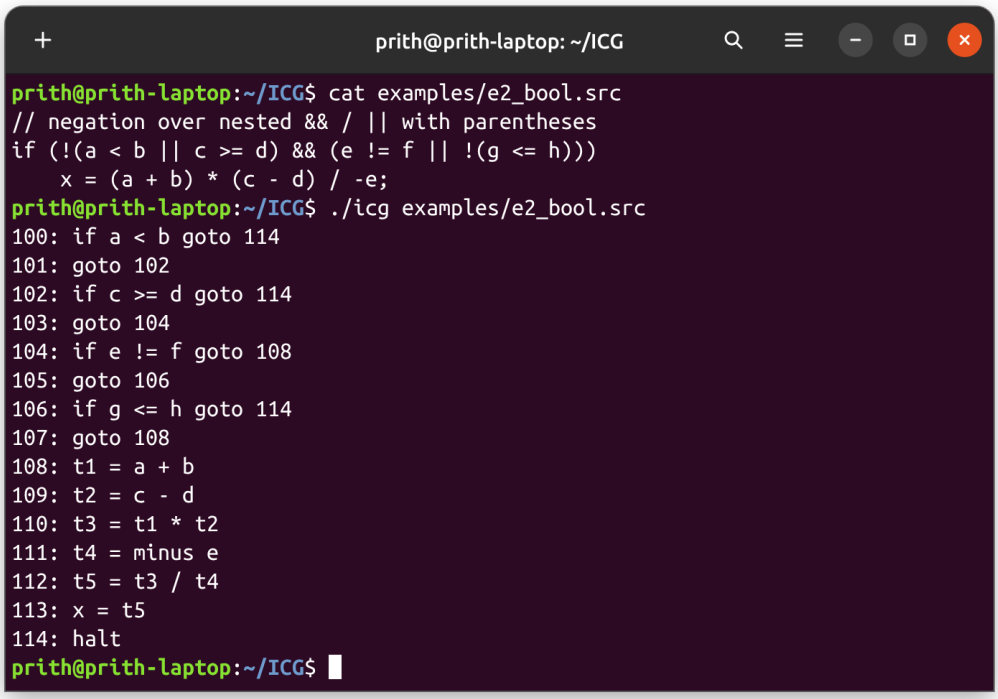


Figure 3.3 Negated disjunction inside a conjunction (`examples/e2_bool.src`)

Because of the outer `!`, the true exits of `a < b` and `c >= d` (quads 100 and 102) become false exits of the whole condition and jump past the assignment to 114. Only when both tests fail does control reach 104, the right operand of `&&`. There a true `e != f` enters the body at 108 without testing `g <= h`, and the inner `!` sends a true `g <= h` to the exit. The body evaluates `(a + b) * (c - d) / -e` with the correct precedence and associativity.

3.4 Nesting

Nesting needs no extra mechanism. Every rule reads only the lists of its own children and hands one `nextlist` up to its parent, so a construct behaves the same at any depth. Statements in sequence are joined by `L → L1 M S`, which fills the previous statement's `nextlist` with the first quad of the next one.

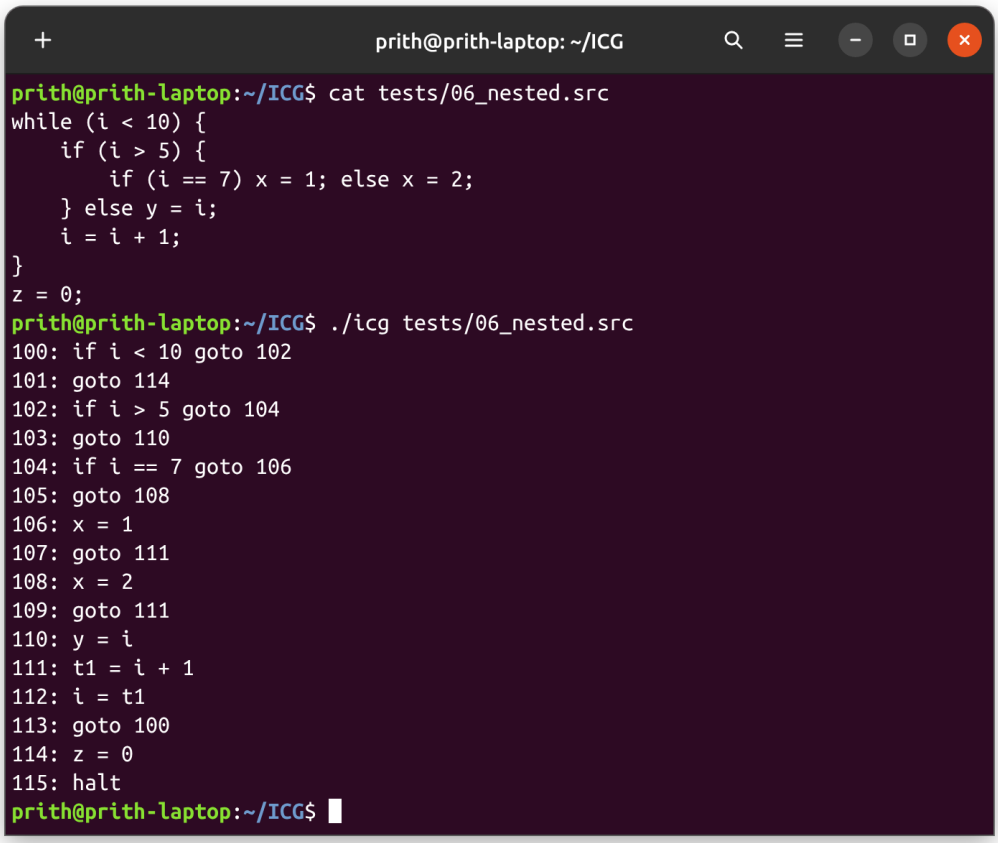


Figure 3.4 if-else inside if-else inside while (`tests/06_nested.src`)

The jumps that end the inner then- and else-branches (107 and 109) both go to 111, where `i = i + 1` begins; the outer else-branch `y = i` at 110 simply falls through to it. Quad 113 returns to the loop test, and the loop's false exit 101 goes to `z = 0` at 114. The same rules handle the dangling else: in `if (a < b) if (c < d) x = 1; else x = 2;` the `else` is attached to the inner `if` (regression test `15_dangling_else`).

3.5 Error reporting

Errors stop compilation with the line number, a message, a non-zero exit status and no code output. Every token has a readable name in the `%token` declarations, for example `RELOP "relational operator"`, and Bison is configured with `parse.error verbose`, so syntax errors name the tokens that were expected.

Table 3.1 Errors detected by the generator

Kind	Detected by	Message form	Exit
Lexical	Flex catch-all rule	`line N: unexpected character 'c'`	1
Syntax	Bison, verbose errors	`line N: syntax error, unexpected X, expecting Y`	1
Misplaced jump	Loop stack	`line N: break outside a loop`	2

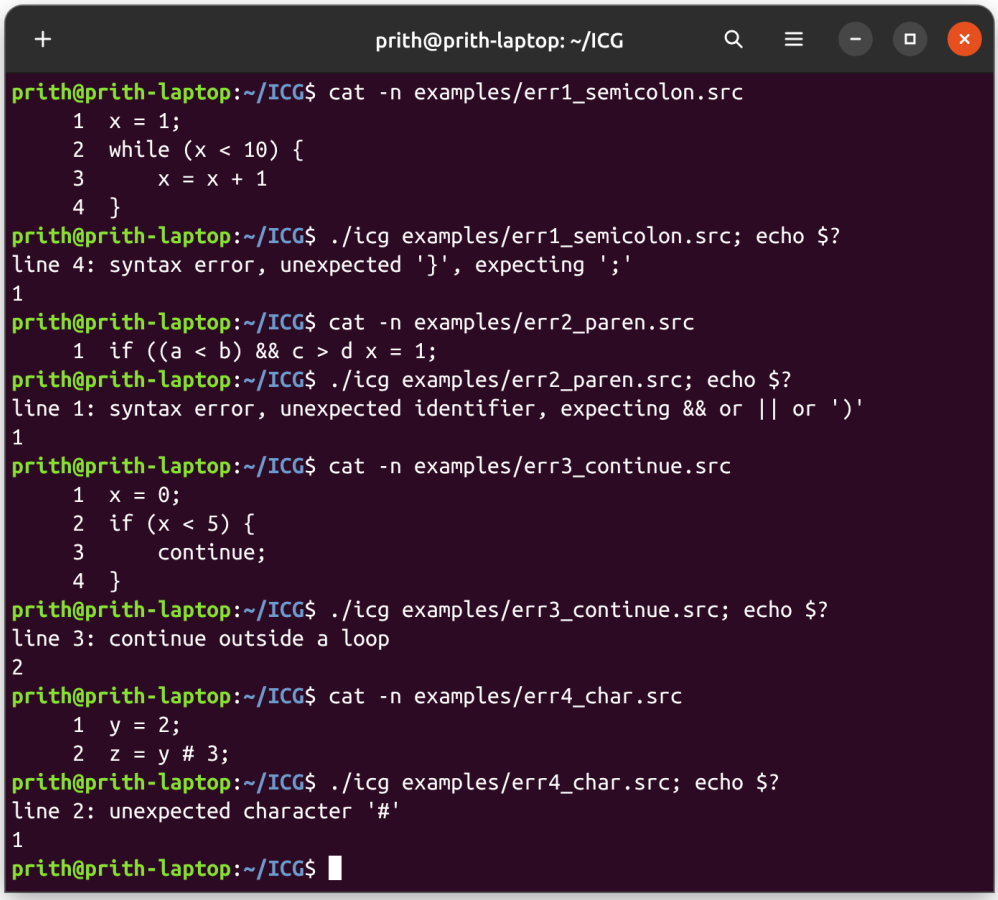


Figure 3.5 Syntax, semantic and lexical errors with line numbers and exit codes

In the first program the missing semicolon after `x = x + 1` is reported on line 4, where the parser meets the unexpected `}`; an LR parser detects a missing token only at the next token it reads. In the second, the three tokens that could legally follow `c > d` point directly at the missing parenthesis.

4 Novelty: Making Backpatching Visible

The project's contribution beyond the textbook scheme is a trace mode that exposes the patch lists while code is generated, addressing the gap identified in Section 2.1. With `--trace`, the generator prints every reduction that emits a jump or changes a patch list, followed by each operation that reduction performs, and then the finished code after a blank line. A jump can be followed from the moment it appears as `goto _` to the `backpatch` line that fills it.

The trace is built into `parser.y` itself, through a few lines in `emit`, `makelist`, `merge` and `backpatch`. It needs no separate program, works in any terminal, and does not change the generated code: for every regression and example program that compiles, the code printed after the trace is identical to the code printed without it.

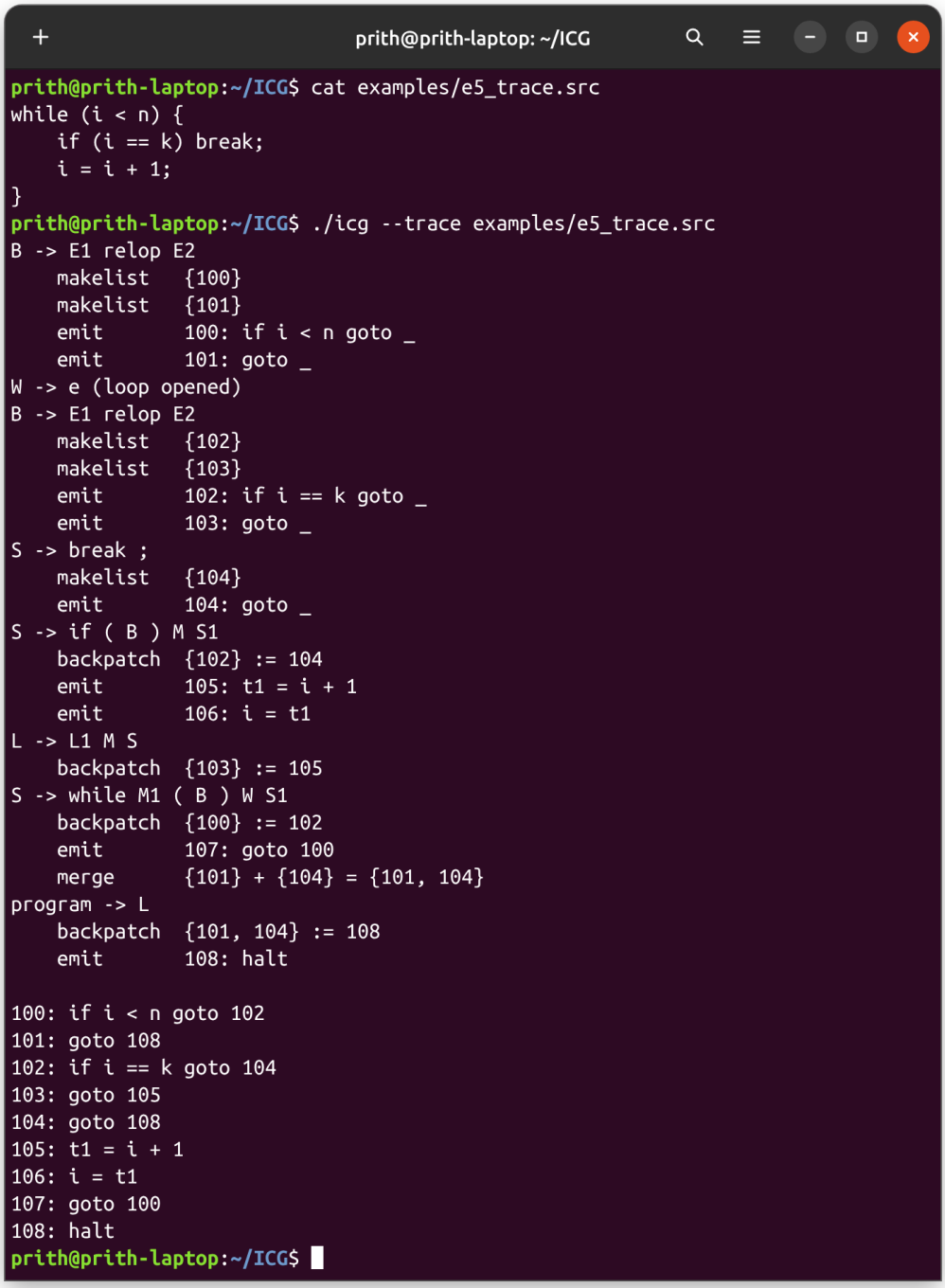


Figure 4.1 Trace of a while loop with a break (`examples/e5_trace.src`)

Reading Figure 4.1 from top to bottom gives the life of every jump, collected in Table 4.1. Quad 104, the `break`, stays empty for the whole body, is merged with the loop's false exit 101 when the loop closes, and both are filled by a single `backpatch` at the end of the program. Quad 107 never enters a list, because its target was known when it was emitted.

Table 4.1 Life of each jump in Figure 4.1

Quad	Emitted by	Filled by	Target
100	B → E1 relop E2 (true exit)	while rule	102, loop body
101	B → E1 relop E2 (false exit)	program → L, after merge with 104	108, after the loop
102	B → E1 relop E2 (true exit)	if rule	104, the `break`
103	B → E1 relop E2 (false exit)	L → L1 M S	105, next statement
104	S → break ;	program → L, after merge with 101	108, after the loop
107	while rule (`emit_goto`)	known when emitted	100, loop test

For teaching, this turns backpatching from a description in a textbook into something that can be watched on a real program: why a list exists, when it grows by `merge`, and which reduction finally fills it. Tools such as JFLAP [6] have shown the value of step-by-step visualisation for automata; the trace brings the same idea to intermediate code generation. A step-by-step variant that pauses after each operation is scheduled for Review 3.

5 Testing

5.1 Build

The grammar is built with `%expect 0`, so Bison stops with an error if any shift/reduce or reduce/reduce conflict exists, and the generated C is compiled with `-Wall -Wextra`. Figure 5.1 is a clean build from source with no conflict reports and no compiler warnings.

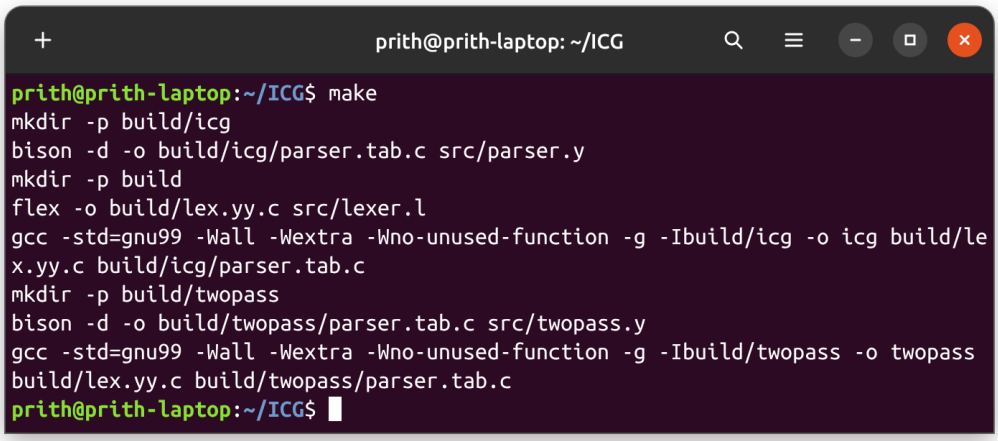


Figure 5.1 Clean build

5.2 Regression suite

Each program in `tests/` has a hand-checked `.expected` file holding the exact standard output and standard error. `run_tests.sh` compiles every program, compares the output with the expected file, and fails any test whose final code still contains `goto _`, even if the output matches. Table 5.1 lists the tests that cover the Review 1 and Review 2 scope.

Table 5.1 Regression tests for the Review 1 and Review 2 scope

Test	What it checks	Result
`01_assign`	Arithmetic, precedence, unary minus, temporaries	Pass
`02_if`	`if` without `else`	Pass
`03_if_else`	`if-else`, the jump over the else-branch	Pass
`04_while`	`while`, backward jump	Pass
`05_bool`	`&&`, `||`, `!` with short-circuit jumps	Pass
`06_nested`	`if-else` in `if-else` in `while`	Pass
`08_syntax_error`	Syntax error naming the expected tokens	Pass
`11_true_false`	Constant conditions `true` and `false`	Pass
`13_lex_error`	Unexpected character rejected	Pass
`15_dangling_else`	`else` binds to the nearest `if`	Pass
`16_trace`	`--trace` output for an `if-else`	Pass
`17_quad_format`	`--format=quad` table	Pass

The full run in Figure 5.2 also contains the tests for the Review 3 constructs and the two-pass cross-check (marked `two-pass`); these are presented in the Review 3 report. All 35 checks pass.

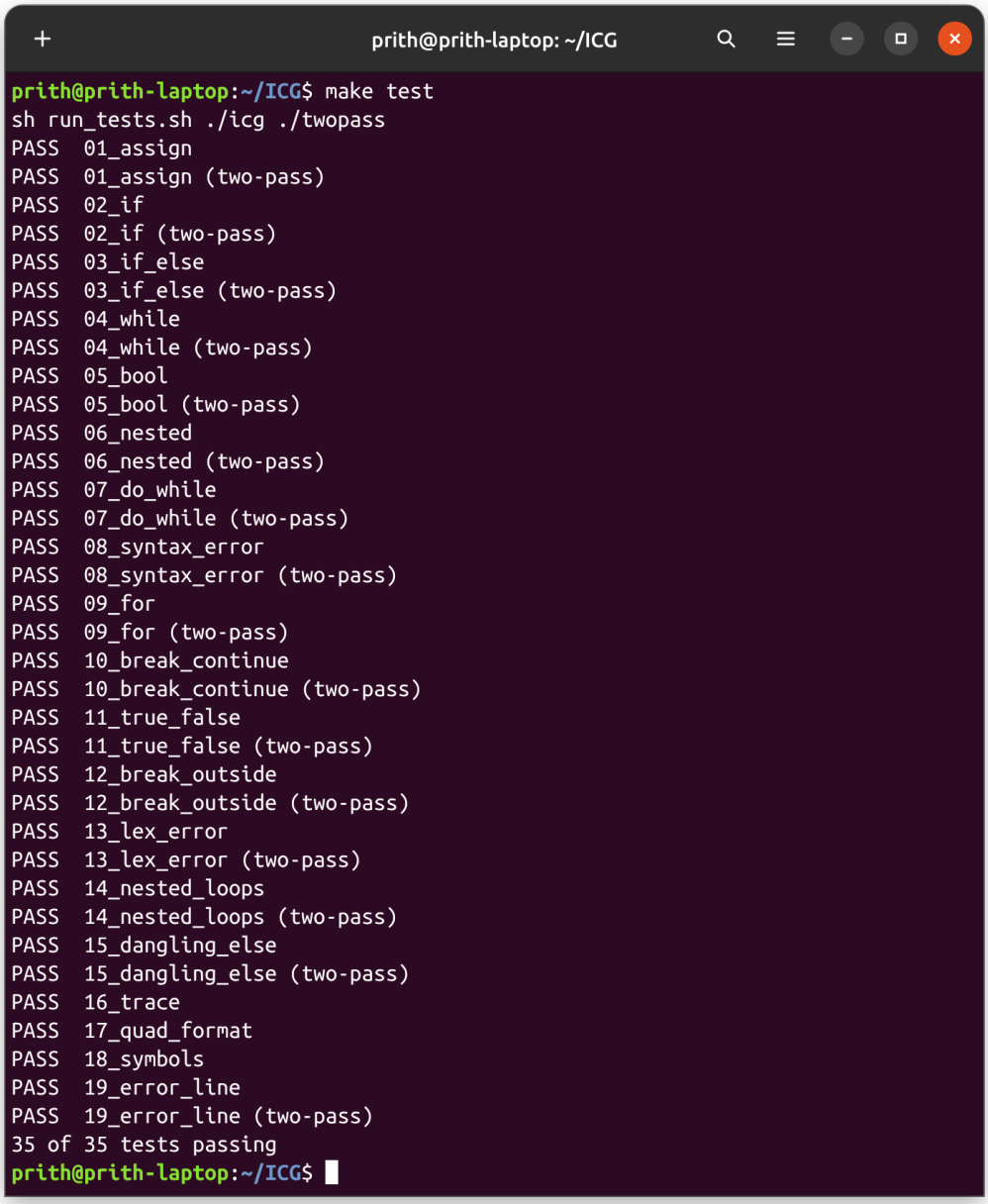


Figure 5.2 Regression run

5.3 Stress programs

Table 5.2 Stress programs for the Review 2 constructs

Program	What makes it hard	Quads	Shown in	Result
`e1_ladder`	Four-level else-if chain; `&&` and `||` in different branches; three `N` jumps pending together	20	Figure 3.1	Correct
`e2_bool`	`!` over a parenthesised `||` inside `&&`, a second `!` in the right operand; unary minus in the body	15	Figure 3.3	Correct
`e5_trace`	Loop exit merged with the loop's false exit; traced	9	Figure 4.1	Correct
`err1`–`err4`	Missing semicolon, unclosed parenthesis, `continue` outside a loop, invalid character	–	Figure 3.5	Correct

6 Progress Summary and Plan for Review 3

6.1 Progress against the plan

Table 6.1 Implementation status at Review 2

Item	Planned for	Status
Lexer, grammar, quad table, patch lists	Review 1	Complete
Assignment, arithmetic, `if`	Review 1	Complete
`if-else` with dangling-else resolution	Review 2	Complete (Section 3.1)
`while`	Review 2	Complete (Section 3.2)
`&&`, `||`, `!`, `true`, `false`	Review 2	Complete (Section 3.3)
Nesting to any depth	Review 2	Complete (Section 3.4)
Error reporting with line numbers	Review 2	Complete (Section 3.5)
`--trace` mode	Review 2	Complete (Chapter 4)
`do-while`, `for`, `break`, `continue`	Review 3	Implemented ahead of schedule; to be demonstrated at Review 3
Step-by-step trace	Review 3	Implemented ahead of schedule; to be demonstrated at Review 3
Two-pass baseline and comparison	Review 3	Implemented ahead of schedule; to be demonstrated at Review 3
Manuscript	Review 3	Draft prepared; to be finalised for Review 3

Every item planned for Review 1 and Review 2 is complete. The remaining constructs are already implemented and pass their tests, so implementation progress is ahead of the 75% expected at this review. The grammar builds with no conflicts, and all regression tests pass.

6.2 Plan for Review 3 (23 October)

- **`do-while` and `for`:** the `for` loop's step is written before its body but runs after it, so the code must be laid out as init, test, step, jump to test, body, jump to step.
- **`break` and `continue`:** a loop stack opened by the marker `W` collects these jumps and fills them when the loop closes; a jump outside any loop is reported as an error.
- **Step mode:** `--step` pauses the trace after every operation, for live walkthroughs.
- **Two-pass baseline:** a second Bison grammar generating the same code through a syntax tree and symbolic labels, to cross-check every semantic action and compare memory use.
- **Manuscript:** the final project report and presentation for submission on 30 October.

References

1. A. V. Aho, M. S. Lam, R. Sethi, J. D. Ullman. *Compilers: Principles, Techniques, and Tools*, 2nd ed. Addison-Wesley, 2006. Ch. 6, sec. 6.6–6.7.
2. A. W. Appel. *Modern Compiler Implementation in C*. Cambridge University Press, 1998. Ch. 7.
3. K. D. Cooper, L. Torczon. *Engineering a Compiler*, 2nd ed. Morgan Kaufmann, 2011. Ch. 7.
4. J. Levine. *flex & bison*. O'Reilly Media, 2009.
5. C. Lattner, V. Adve. "LLVM: A Compilation Framework for Lifelong Program Analysis & Transformation." *Proc. International Symposium on Code Generation and Optimization (CGO)*, 2004, pp. 75–86.
6. S. H. Rodger, T. W. Finley. *JFLAP: An Interactive Formal Languages and Automata Package*. Jones and Bartlett, 2006.